AWS Streaming Data Lakehouse: System Architecture & Technical Documentation

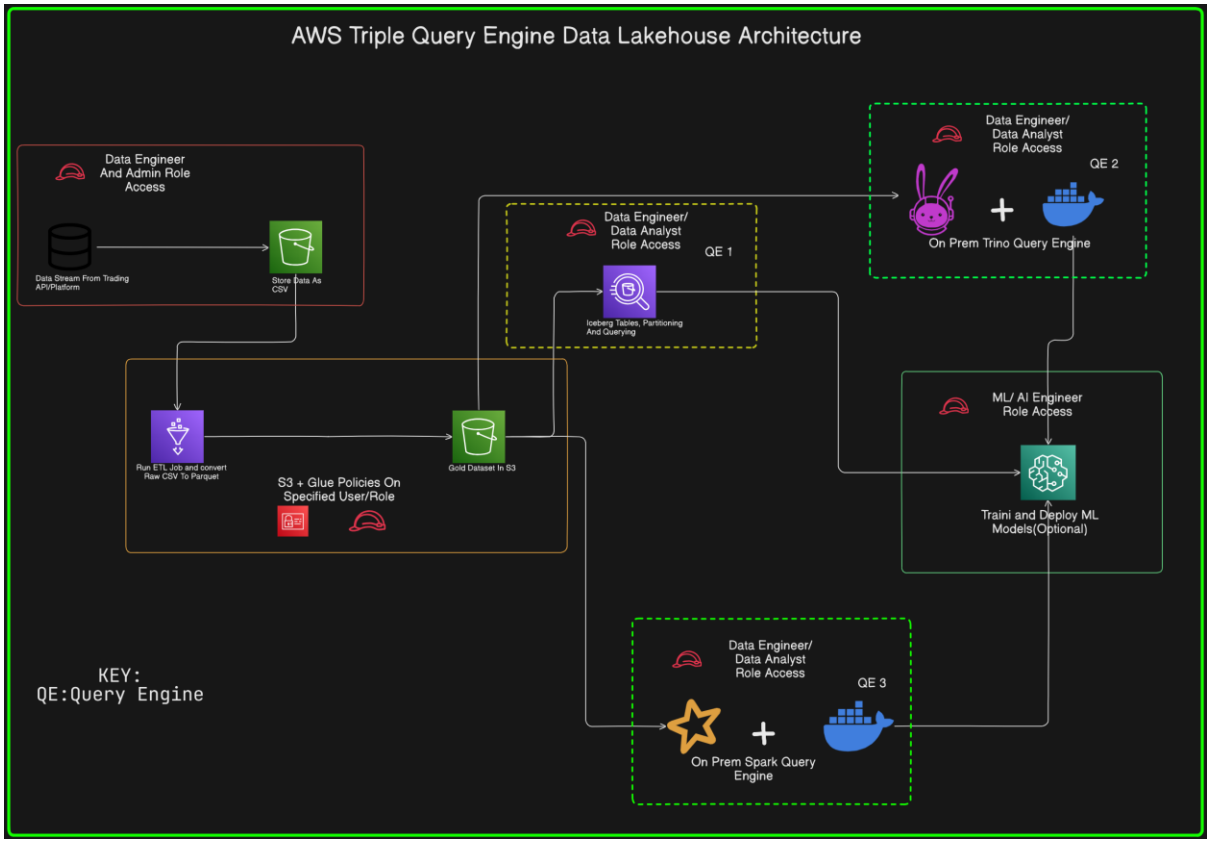
Foreword

This document provides a high-level introduction and structural overview of the AWS Data Lakehouse (Medallion Architecture), with a Triple Query Engine setup.

The system is designed to ingest high-frequency financial tick data, process and store it efficiently, and apply effective query methods and partitioning for fast data access during backtests and walk forward analysis.

This document serves as a guideline to setup a similar environment on you AWS account, provided that you follow all the steps and have the required files in place.

Here is the pipeline diagram for further context:



Tick Data Acquisition

The first stage of the pipeline is responsible for the acquisition of **raw, high-frequency tick data** from the **Dukascopy** market data service using **Quant Data Manager**.

Quant Data Manager is used as a lightweight and straightforward data collection application, providing a simple interface for downloading historical tick data from the Dukascopy website. A **free version** of the software is available, making it suitable for individual research and prototyping use cases. The application is **officially available for Windows platforms**; support for macOS was not a requirement for this pipeline and was not evaluated.

Tick data is collected at the finest available temporal resolution and includes individual market events such as bid and ask price updates, Date and Time, and associated volumes. During the download process, the data is written directly to disk as **CSV files**, with no transformation, aggregation, or filtering applied.

The resulting CSV files represent the **raw, immutable data layer** of the pipeline. Once saved, these files are treated as read-only and serve as the authoritative source for all downstream processing, validation, and analysis. Preserving the data in its original CSV form ensures traceability, reproducibility, and the ability to reprocess the dataset as pipeline logic evolves.

Quant Data Manager is used exclusively for data acquisition, while all subsequent processing, storage optimization, and machine learning workflows are handled in later stages of the system. The application itself is easy to use and setup, if one wishes to collect data the manually way using a web scrapper, best of luck as it will be a very time-consuming task, **QDM** simply makes all this easier. This method is best for bulk data, data that goes a long way back, such as a 20-year period.

For getting **live data**, you will need to connect to your own trading platform and build a script to collect the live ticks on the live/demo account. Some brokers have an API but requires a deposit into your trading account to get it. This is one method; however, we have another effective method for live data.

This method involves us connecting to our brokers account and getting the ticks live as we interact with the charts and backend API. This method will work well on Meta Trader 5 and other trading platforms. Feel free to try other APIs, there is many ways to get this data, I was simply exploring the methods that would work well for this project, without increasing overhead costs.

Script List

Overview

This section consists of a list of script that will be used in this pipeline. They are listed in order of execution, given that you have the correct Role alongside with S3 and Glue Policies. If unsure, contact your AWS Admin and ask that they grant you these Policies. For those of you that are doing this locally, you can create a new user on the AWS root account and assign the user Admin policies and permissions. This is an easy task; a simple YouTube video is all you need to get started. Feel free to watch as many as you like. Please note that this is done on the AWS FREE TIER, so I do not expect you to spend any money at all, UNLESS you're on the paid plan. The setup is cheap, on the paid account, you can expect to spend less than 3 dollars at max.

Okay with that out of the way, let us review the list of scripts that we will be using in this setup:

1.mt5_s3_data_streamer.py

2.csv_to_parquet_glue_job.py

3.iceberg_migration.py

4.schema_evolution.py

5.iceberg_time_travel.py

6.spark_time_travel.py

7.local_trino_time_travel.py

8.docker-compose.yml

9.hive.properties

10.iceberg.properties

11.symbols.json

12.env file 1 for AWS Athena credentials and s3 buckets

13.env file 2 for Spark and Trino credentials as well as AWS bucket credentials

Please verifiy all the files exist to avoid any kind of complications, as all the scripts were meant to be used together.

mt5_to_s3.py

Overview:

This script streams live tick data from MetaTrader 5 (MT5) directly to an AWS S3 bucket. The data is collected during predefined windows and stored in a CSV format. It uploads data to S3 at the end of each collection window, ensuring only the first tick within each second is recorded.

Key Features:

- Uses `MetaTrader5` and `boto3` SDKs to handle MT5 connections and S3 interactions.
- Configurable collection windows for streaming tick data.
- Uploads files with a specific naming convention: `BTCUSD_20260306_0750.csv`.

Usage:

```
python mt5_to_s3.py [--config symbols.json] [--loglevel DEBUG]
```

Functions:

1. **`current_sast()`**: Returns the current time in SAST timezone.
2. **`window_for(now: datetime)`**: Determines if the current time falls within predefined collection windows.
3. **`is_symbol_market_open(symbol: str)`**: Checks if the market for a specific symbol is open.
4. **`get_new_ticks(symbol: str)`**: Fetches new ticks from MT5 since the last valid poll.

Environment Variables:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `AWS_REGION`
- `BUCKET`

csv_to_parquet_glue_job.py

Overview:

This AWS Glue job transforms raw CSV tick data stored in S3 into Parquet files, facilitating more efficient storage and querying.

Key Features:

- Reads data using a specified schema and handles the conversion to Parquet format.
- Supports idempotent file conversions to prevent duplicate processing.
- Utilizes `boto3` for S3 interaction and `pyspark` for data processing.

Usage: Run as a Glue Job within AWS.

Functions:

1. **`list_csv_files(s3_client, bucket, prefix)`**: Retrieves a list of CSV files from a specific S3 prefix.
2. **`csv_key_to_parquet_key()`**: Converts a CSV key to a corresponding Parquet key.
3. **`parquet_exists(s3_client, bucket, key)`**: Checks if the Parquet file already exists in S3.

iceberg_migration.py

Overview:

This script facilitates the migration of tick data into Iceberg tables, allowing for efficient data management and schema evolution.

Key Features:

- Creates database and external tables in AWS Glue for raw data.
- Supports upsert operations to avoid duplication.
- Allows for the addition of new columns and table properties.

Usage: Run from the command line after configuring environment variables.

Functions:

1. **`run_athena_query(query, silent=False)`**: Executes an Athena query and waits for its completion.
2. **`get_count_from_query(query_id)`**: Fetches the result of a COUNT query.

schema_evolution.py

Overview:

This script enables schema evolution for existing Iceberg tables, providing functionality to modify table structures.

Key Features:

- Supports adding, dropping, and modifying columns within Iceberg tables.
- Allows for partition specification adjustments.

Usage: Executed from the command line; requires configured environment.

Functions:

1. **`add_column(table, column_name, column_type)`**: Adds a new column to the specified table.
2. **`drop_column(table, column_name)`**: Drops a column from the specified table.
3. **`set_partition_spec(table, spec)`**: Modifies the partition specifications for a table.

iceberg_time_travel.py

Overview:

This script allows users to explore historical versions of Iceberg tables using snapshot IDs or timestamps.

Key Features:

- Provides the ability to view snapshots, partitions, and sample the data at specific points in time.

Usage: Run from the command line.

Functions:

1. **`list_iceberg_tables()`**: Lists all Iceberg tables available within a given schema.
2. **`time_travel_by_snapshot(table, snapshot_id)`**: Reads the data from an Iceberg table at a specific snapshot ID.

spark_time_travel.py

Overview:

This script is designed to interface with Spark to explore Iceberg tables and their history.

Key Features:

- Provides a menu for various operations including querying tables and viewing snapshots.
- Supports actions such as rolling back to a snapshot or adding columns.

Usage: Run as a Python script from the command line.

Functions:

1. **`show_snapshots(table)`**: Displays all snapshots for a specified Iceberg table.

2. **`rollback_to_snapshot(table, snapshot_id)`**: Rolls back a table to a specified snapshot.

local_trino_time_travel.py

Overview:

This script connects to a local Trino instance to manage Iceberg tables.

Key Features:

- Allows execution of SQL queries against Iceberg tables.
- Provides functions for querying historical data via snapshots.

Usage: Run from the command line; requires proper configuration of connection details.

Functions:

1. **`get_snapshots(table)`**: Retrieves and displays snapshots for a specified table.
2. **`query_sample(table, snapshot_id=None, timestamp=None, limit=10)`**: Queries a sample of data based on a specified snapshot or timestamp.

docker-compose.yml

Overview:

Configuration file for Docker services including Trino and Spark, facilitating the setup of an isolated environment for data processing.

Key Features:

- Defines services for Trino and Spark with appropriate port mappings and environment variables.
- Includes configuration for connecting to AWS S3 with credentials.

Configuration Files

- **`2.env files`**: Environment variables for configuring access to AWS services, including credentials and region settings. One env file sits in the Query Engine Setup folder, and the other in the main directory AWS Data Lakehouse. Refer to the environment variables placeholder scripts, it contains all the variables that you need.

- **`symbols.json`**: Configuration file specifying trading symbols and settings for the MT5 data streamer.

Conclusion

The integrated scripts provide a comprehensive setup for managing an Iceberg data lakehouse. Key functionalities include schema management, data loading, and snapshot-based querying. Each component is designed to work seamlessly with AWS Athena, Spark and Trino.

The provided documentation ensures that users and developers have a clear understanding of the code's purpose, functionality, and usage, enabling effective integration and maintenance.

This concludes the end of the documentation, for further feedback and assistance you are more than welcome to contact me.

https://www.linkedin.com/in/nokul-debanath-750a56352

AlgoDeveloper400

https://www.kaggle.com/nokuldebanath

https://www.youtube.com/@BDB5905